

[Guides](#) / [High availability](#) / [Multi cluster v2](#) / Multi-cluster deployments (v2)

Multi-cluster deployments (v2)

26.7.0

Connect multiple Keycloak deployments without an external Infinispan cluster



This guide is describing a feature which is currently in preview. Please provide your feedback by [joining this discussion](#) while we're continuing to work on this.

Keycloak allows deployments that consist of multiple Keycloak instances that connect to each other using its embedded Infinispan caches. Load balancers can distribute the load evenly across those instances. Such setups are intended for transparent networks; see [Single-cluster deployments](#) for more details.

A multi-cluster setup adds additional components to provide high availability that may be needed for some environments.

Unlike the standard [Multi-cluster deployments \(v1\)](#), it removes the requirement for an external Infinispan cluster. This simplifies the deployment architecture and lifts any requirement for a specific environment such as Kubernetes or AWS.

When to use a multi-cluster setup

The multi-cluster deployment capabilities of Keycloak are targeted at use cases that:

- Are constrained to a single region or an equivalent low-latency setup.
- Permit planned outages for maintenance.
- Fit within a defined user and request count.
- Can accept the impact of periodic outages.
- Provide data centers with the appropriate network latency and database configuration.

Tested Configuration

We regularly test Keycloak with the following configuration:

- Two OpenShift single-AZ clusters, in the same AWS Region
 - Provisioned with [Red Hat OpenShift Service on AWS](#) (ROSA), using ROSA HCP.
 - All worker nodes reside in a single Availability Zone.
 - OpenShift version 4.21.
- Amazon Aurora PostgreSQL database
 - High availability with a primary DB instance in one availability zone, and a synchronously replicated reader in the second availability zone.
 - Version 17.5
- AWS Global Accelerator, sending traffic to both ROSA clusters

Infrastructure requirements

The following infrastructure requirements need to be met:

- A highly available database with synchronous replication.
- Two or more clusters with low latency for read, write, and commit operations to the database. A latency of less than 5 ms is suggested, and below 10 ms is required.

On this page

[When to use a multi-cluster setup](#)

[Tested Configuration](#)

[Infrastructure requirements](#)

[Tested load](#)

[Limitations](#)

[Next steps](#)

[Concept and building block overview](#)

[Example blueprint](#)

[Operational procedures](#)

 [Edit this guide](#)

- A round-trip latency below 5 ms is suggested, and below 10 ms is required, within each cluster.
- An external load balancer to balance requests between the clusters.

Latency and latency spikes amplify in the response time of the service and can lead to queued requests, timeouts, and failed requests. Networking problems can cause downtimes until the failure detection isolates problematic nodes.

While equivalent setups should work, you will need to verify the performance and failure behavior of your environment. We provide functional tests, failure tests and load tests in the [Keycloak Benchmark Project](#).

Tested load

We regularly test Keycloak with the following load:

- 1,000,000 users
- 300 requests per second



It is imperative that your production deployments are integrated with an observability stack in order to identify issues early and facilitate troubleshooting when they do arise. Additional demand on Keycloak makes isolating issues harder, therefore this becomes increasingly pertinent as the total number of users and requests per second increases.

While we did not see a hard limit in our tests with these values, we ask you to test for higher volumes with horizontally and vertically scaled Keycloak instances and databases.

Limitations

- During certain failure scenarios, there may be downtime of several minutes depending on the configuration of the database, the load balancer, or Keycloak. By default, Keycloak detects the failure of another node within one minute and either isolates it from the cluster or propagates its state via readiness probes to the load balancer.
- After certain failure scenarios, manual intervention may be required depending on the database vendor.

Next steps

The different guides introduce the necessary concepts and building blocks. For each building block, a blueprint shows how to set up a fully functional example. Additional performance tuning and security hardening are still recommended when preparing a production setup.

Concept and building block overview

- [Concepts for multi-cluster deployments \(v2\)](#)

Example blueprint

While this blueprint uses Kubernetes, the concepts are not limited to Kubernetes.

- [Deploying Keycloak for HA with the Operator \(v2\)](#)

Operational procedures

- [Managing upgrades \(v2\)](#)

Keycloak is a Cloud Native Computing Foundation incubation project



